Strategic Remediation Plan for Doctoral Thesis: Engineering Dynamic Succinct Data Structures

1. Diagnostic Assessment of Examination Outcomes

The examination of the doctoral thesis, "Engineering Dynamic Succinct Data Structures," conducted in October 2025, resulted in a recommendation for major amendments with a resubmission window of six months.[1] A forensic analysis of the examiners' joint report reveals a fundamental dichotomy in the evaluation: while the scientific contribution—specifically the development of novel dynamic data structures for integer compression—is acknowledged as "distinct," "original," and "worthy of publication," the presentation and methodological rigor fall below the requisite standard for a doctoral qualification.[1]

The examiners have identified that the candidate has conducted a "methodical study" [1], yet the thesis fails to frame this engineering effort within a systematic research methodology that includes rigorous quantitative validation. The feedback can be categorized into four critical deficits:

1. **Methodological Deficit:** The absence of explicit, quantified research questions and success criteria.
2. **Theoretical Deficit:** A lack of formal verifiability analysis (correctness proofs) and rigorous Information Theoretic Lower Bound (ITLB) efficiency analysis in dynamic settings.
3. **Comparative Deficit:** Insufficient benchmarking against the current State-of-the-Art (SOTA), specifically the 2021 work of Dinklage, Fischer, and Herlez.[2]
4. **Presentation Deficit:** Stylistic issues regarding the use of active voice ("we/our") and the organization of chapters.[1]

This report establishes a comprehensive strategic roadmap to address these deficiencies. It transforms the examiner's feedback from a list of corrections into a structured engineering and writing program. The objective is to elevate the thesis from a report on software development to a rigorous scientific treatise on algorithm engineering.

1.1 Decomposition of Examiner Requirements

To ensure total compliance, the feedback has been deconstructed into actionable work packages. The following table maps the specific examiner comments to the strategic response required.

Examiner Feedback	Core Issue	Strategic Response Strategy
"Reorganise chapters... demonstrate research was systematically carried out."	Narrative Structure	Implement a dedicated **Methodology Chapter**; restructure contribution chapters to follow a Design-Analysis-Evaluation pattern.
"Explain in detail what research method and validation method were used."	Methodology	Define the work as **Algorithm Engineering**; detail the experimental cycle and verification logic.
"Define a research question quantitatively... State clearly what quantified success criteria are."	Precision	Formulate RQs based on specific latency/space bounds (e.g., <2 bits overhead, $4\times$ speedup).
"Clarify why solving this question will help real applications... big-data/cloud."	Relevance	Add specific use-cases: Inverted Index compression, Cloud Storage Deduplication, Genomic Indexing.
"Prove the correctness of proposed implementations/algorithms."	Rigor	Develop formal **Verifiability Analysis** (Hoare Logic/Invariants) for SIMD algorithms.
"Demonstrate how efficient... achieving information theoretic lower bound... in dynamic setting."	Theory	Perform explicit **Gap Analysis** ($Space_{actual} - ITLB$) including dynamic resizing overhead (f-factor).
"Consider more recent work such as Patrick ...	Benchmarking	Execute direct benchmarks against **Dinklage et al.**

detailed comparative analysis."		**(2021)**; analyze trade-offs between B-trees and SIMD arrays.
"Use passive voice instead of 'we' and 'our'."	Style	Complete stylistic overhaul of the text; shift focus to the artifacts and results.
"Include user interface of your prototype."	Evidence	Develop a visualizer tool for internal data states; include screenshots in Chapter 5 or Appendices.

1.2 The Strategic Pivot: From Implementation to Validation

The central theme of the revision must be a shift in perspective. The original thesis presented *what was built*. The revised thesis must present *proven knowledge*. The examiners explicitly requested the inclusion of lines of code and sample data [1], but paradoxically also demanded high-level theoretical proofs. This suggests the examiners view the current work as a "black box." They see the results but do not trust the mechanism because the mechanism has not been mathematically verified or systematically compared.

The remediation plan prioritizes **Transparency** and **Verifiability**. Every algorithm presented in Chapters 3, 4, and 5 will undergo a rigorous "white-box" analysis where its internal logic is proven correct before its performance is measured.

---

2. Theoretical Framework and Methodological Reconstruction

The absence of a dedicated methodology chapter is a critical structural flaw identified by the examiners.[1] To address this, a new **Chapter 3: Research Methodology and Theoretical Framework** must be authored. This chapter will bridge the gap between the background (Chapter 2) and the technical contributions.

2.1 Defining the Research Paradigm

The thesis operates within the paradigm of **Algorithm Engineering**. This methodology is distinct from pure theoretical algorithm design (which focuses solely on asymptotic complexity) and software engineering (which focuses on robustness). Algorithm Engineering iterates through *Design*, *Analysis*, *Implementation*, and *Experimentation*.

The revised thesis must explicitly state that the research method employed is **Experimental Algorithmics**. This involves:

1. **Hypothesis Formulation:** Deriving theoretical performance expectations based on memory hierarchy and instruction sets (SIMD).
2. **Implementation:** Engineering the data structures (EA, Predecessor, Prefix Sum) using low-level optimizations (AVX2/AVX-512).
3. **Verification:** Proving the implementation matches the theoretical specification (Verifiability Analysis).
4. **Evaluation:** Measuring performance against established baselines (STL) and SOTA (Dinklage) under controlled conditions.

2.2 Quantified Research Questions and Success Criteria

The examiners demand "quantified" criteria.[1] Vague goals like "faster search" or "less memory" are insufficient. The new Introduction and Methodology chapters must enshrine specific targets derived from the limitations of current systems.

2.2.1 Formulating Quantified Research Questions

- **RQ1 (Dynamic Space Efficiency):** "Can an append-only Extensible Array (EA) achieve a worst-case storage redundancy of $r(n) \le \frac{n}{w} + O(\sqrt{n})$ bits over the Information Theoretic Lower Bound (ITLB) during dynamic growth, while supporting $O(1)$ amortized access time?"
- **RQ2 (Small-Set Latency):** "Is it possible to reduce the instruction count for predecessor search in small sets ($|S| \le 128$) by a factor of 2 compared to branch-dependent binary search, using SIMD minpos intrinsics, thereby achieving query latencies < 50 cycles?"
- **RQ3 (Update Efficiency):** "Can a Dynamic Searchable Prefix Sum structure support updates in amortized constant time $O(1)$ with a synchronization overhead of $< 5\%$ compared to the static query time?"

2.2.2 Defining Success Criteria

The thesis will claim success if and only if the following metrics are met (and proven in the Evaluation chapters):

1. **Space Success:** The Dynamic Elias-Fano structure must consume less than $1.1 \times$ the space of a static representation for the same data, verifying the efficacy of the resizing strategy.
2. **Latency Success:** The SIMD Predecessor Search must outperform the C++ Standard Template Library (std::set lower_bound) by at least 300% (4x speedup) for sets fitting in L1 cache.[4]
3. **Correctness Success:** All algorithms must pass a formal verifiability check where the pre-conditions and post-conditions are mathematically demonstrated to hold for all valid inputs.

2.3 The "Computing Environment" Definition

To satisfy the requirement to "Define a research question quantitatively, including the computing environment" [1], a rigorous definition of the experimental testbed is required. This is not merely a list of hardware but a definition of the *constraints* under which the algorithms are optimized.

Computing Environment Specification:

- **Architecture:** x86-64 with AVX2 support (Essential for Chapter 4/5).
- **Memory Hierarchy:**
  - L1 Instruction/Data Cache: 32KB+ (Latency ~4 cycles).
  - L2 Cache: 256KB+ (Latency ~12 cycles).
  - L3 Cache: Shared 8MB+ (Latency ~40 cycles).
- **Justification:** The succinct data structures are designed to reside primarily in L1/L2 cache. The research question effectively asks: "Can succinct compression maximize effective L1 cache capacity?"

---

3. Correctness and Verifiability Analysis

A major new requirement is the "Verifiability Analysis" of the algorithms.[1] This requires translating the C++ code into logical proofs. This is particularly challenging for SIMD instructions, which perform parallel operations that are opaque to standard sequential logic.

3.1 Mathematical Verifiability Framework

For each core algorithm (Extensible Array Resize, Predecessor Rank, Prefix Sum Update), the thesis must provide a proof of correctness. This should follow the structure of Hoare Triples $\{P\} C \{Q\}$, where P is the precondition, C is the command (algorithm), and Q is the postcondition.[5]

3.1.1 Proof Strategy for Extensible Arrays (Chapter 3)

- **Algorithm:** GROW().[1]
- **Invariant:** For an array A of size n and capacity C, valid elements exist at indices $0 \dots n-1$.
- **Proof Obligation:** Show that after GROW():
  1. New capacity $C' = C + f(C)$.
  2. $\forall i \in [0, n-1], A'[i] = A[i]$.
  3. The memory range $[0, C'-1]$ is allocated and accessible.
- **Verifiability Argument:** The thesis must verify that the copy operation (lines 3-5 in Algorithm 1) preserves data integrity. For bit-packed arrays (width $w < 64$), this involves proving that bit-shifting operations correctly align elements across word boundaries.

- *Proof Sketch:* Let element e_i reside at bit offset $o_i = i \times w$. Let word index $W_i = \lfloor o_i / 64 \rfloor$ and bit index $b_i = o_i \pmod{64}$. The retrieval function $Access(i)$ reconstructs e_i from W_i and W_{i+1}. The GROW function performs a memcpy or element-wise copy. Since memcpy preserves bit-order, and the logical indexing formula relies solely on i and w (which are invariant), the structure preserves data integrity.

3.1.2 Proof Strategy for SIMD Predecessor Search (Chapter 4)

This is the most complex verification task. The thesis proposes an "Improved Fredman" method using _mm256_minpos_epu16.[1] The examiner explicitly asks to "Prove the correctness of proposed implementations".[1]

The Algorithm Logic [1]:

1. Broadcast search key x to vector V_x.
2. Load keys S to vector V_S.
3. Compute $V_{diff} = V_x \oplus V_S$ (XOR).
4. Compute $V_{min} = \text{minpos}(V_{diff})$.
5. Extract index i of the minimum XOR value.
6. Claim: $S[i]$ has the longest common prefix (LCP) with x.

The Verification Gap:
Does minimizing the XOR value guarantee finding the element with the Longest Common Prefix?

- *Analysis:* $A \oplus B$ results in a bit-string where 1s indicate differences. The numeric value of $A \oplus B$ is dominated by the Most Significant Bit (MSB) of the difference.
- *Proof:*
  - Let k be the position of the most significant difference between x and $S[i]$.
  - The value of the XOR difference is roughly 2^k.
  - Minimizing the XOR value minimizes 2^k, which implies minimizing k.
  - A smaller k means the difference occurs at a *lower* bit position (less significant).
  - *Correction/Refinement:* Actually, we want to maximize the LCP, which means the difference should be at the *lowest possible* bit position? No, LCP means matching high bits. If high bits match, the difference is at a low bit position, so the XOR value is *small*.
  - *Conclusion:* Yes, minimizing the numeric value of the XOR difference ($x \oplus S[i]$) selects the element where the mismatch occurs at the least significant bit possible, implying the longest matching prefix of high-order bits.
- **Formal Proof Statement:**
  - Let $LCP(a, b)$ be the length of the common prefix.
  - Let $Diff(a, b) = a \oplus b$.
  - $LCP(a, b) = \ell \iff \lfloor \log_2 (Diff(a, b)) \rfloor = w - 1 - \ell$.
  - Therefore, minimizing $Diff(a, b)$ is equivalent to minimizing the position of the MSB

of the difference, which is equivalent to maximizing ℓ (the LCP length).
    - *Verifiability:* The _mm256_minpos_epu16 instruction finds the minimum unsigned 16-bit integer. This hardware instruction is formally specified by Intel to satisfy $\min(V)$. Thus, the composition of XOR and MINPOS correctly identifies the LCP candidate.

This level of detail is exactly what the examiners are missing. The revised Chapter 4 must contain a section "4.X Correctness Proof of SIMD LCP Extraction" detailing this logic.

---

4. Information Theoretic Lower Bound (ITLB) in Dynamic Contexts

The examiners require a demonstration of efficiency relative to the ITLB "especially in the context of dynamic setting".[1] This addresses the core tension in succinct data structures: the trade-off between space compactness and update flexibility.

4.1 Static vs. Dynamic ITLB

The standard ITLB for storing a set of n elements from universe u is $Z = \lceil \log_2 \binom{u}{n} \rceil$.[6]

- Static succinct structures achieve $Z + o(Z)$.
- Dynamic structures introduce "gaps" or "capacity overhead."

4.2 Analyzing the Extensible Array (Chapter 3)

The thesis proposes a resizing strategy that is not simple doubling ($2n$) but uses a factor f.

- **Analysis:** Let current size be n and capacity be C. The space usage is $C \times w$ bits.
- **Redundancy (r):** The "wasted" space is $(C-n) \times w$.
- **Dynamic Behavior:** In the worst case (just after a resize), $C \approx n(1 + 1/f)$.
- ITLB Efficiency Claim:

  $$Space_{dynamic}(n) = Z + \text{structural_overhead} + \text{dynamic_overhead}$$
  $$Space_{dynamic}(n) \approx n \log \frac{u}{n} + O(n) + \frac{n}{f}$$
- **Optimization:** The thesis must demonstrate that by tuning f (e.g., $f = \log n$), the dynamic overhead becomes $o(n)$, thereby maintaining succinctness. The examiners want to see this derivation.

Action: Create a new graph in Chapter 3: "Space Overhead vs. Growth Factor." Plot the theoretical ITLB line and the sawtooth wave of the Extensible Array's actual memory usage.

This visual proof satisfies the requirement to "Demonstrate how efficient... in dynamic setting".[1]

---

5. Comparative Analysis: The "Patrick Dinklage" Benchmark

The most specific technical instruction is to "consider more recent work such as Patrick ... and include a detailed comparative analysis".[1] This refers to the paper **"Engineering predecessor data structures for dynamic integer sets"** by Dinklage et al. (SEA 2021).[2]

5.1 Deconstructing the Competitor

Dinklage's work represents the SOTA for dynamic predecessor search.

- **Technique:** It combines "Universe Reduction" (similar to Van Emde Boas) with "Dynamic Fusion Nodes" (B-trees where nodes are compressed using SIMD).[2]
- **Performance:** It outperforms the C++ STL (std::set) by up to 4x and uses 1/3 of the memory.[4]
- **Hardware:** It utilizes AVX-512 where available, or AVX2.[2]

5.2 The "Gap" in the Current Thesis

The current thesis compares against "Branchless Binary Search" and "Fredman's Method".[1] It ignores the sophisticated engineering of Dinklage. The examiners view this as a critical omission because Dinklage also uses SIMD and addresses the same problem.

5.3 Strategic Positioning for the Benchmark

The student cannot simply run Dinklage's code and hope to win. Dinklage's solution is robust for large sets ($N > 10^6$). The student's solution (Chapter 4) appears optimized for "Small Sets" (e.g., B-tree nodes, $N < 256$) using flat arrays and direct SIMD.

The Benchmark Plan:

1. **Scope:** Focus the comparison on *small set sizes* ($N \in $). This is the "node size" for B-trees, a critical application mentioned in the thesis.[1]
2. **Metrics:** Query Latency (ns) and Construction Time (ns).
3. **Hypothesis:** The student's "Improved Fredman" (Algorithm 7) uses a single AVX2 register broadcast and compare. This is $O(1)$ with a very small constant. Dinklage's Fusion Node involves navigating a compressed trie structure, which—while $O(1)$—has a higher constant due to complexity.
4. **Visual Evidence:** Produce a plot "Figure 4.X: Latency Comparison - Dinklage vs. Improved Fredman." The X-axis should be Set Size (log scale). The Y-axis should be

Latency.
    - *Expectation:* The student's curve should be lower (better) for $N < 64$. Dinklage might win for larger N.
5. **Narrative:** Frame the student's contribution as "The optimal node-level primitive for B-trees," while acknowledging Dinklage as the superior solution for "stand-alone large dynamic sets." This nuance demonstrates the "nuanced understanding" requested.

Implementation Detail:
The student must download the code from Dinklage's repository (linked in 2: github.com/pdinklag/tdc), compile it, and run the predecessor benchmark on the exact same hardware defined in the Methodology chapter.

---

6. Structural Re-engineering and Presentation

The examiners requested a "Reorganise chapters" and "Add a summary for all chapters".[1] They also demanded a shift to **passive voice**.

6.1 Proposed New Structure

The following structure integrates all requirements.

Chapter	Title	Key Revisions & Additions
1	**Introduction**	Add Quantified Success Criteria; Define RQs; Big Data Context.
2	**Literature Review & Background**	Expand ITLB theory; Add section on "Verifiability in Data Structures"; Add deep review of Dinklage .
3	**Methodology & Verification Framework**	**(New)** Define Algorithm Engineering; Computing Env; Proof Logic; Benchmark protocols.
4	**Dynamic Extensible Arrays**	(Was Ch 3) Add ITLB dynamic gap analysis; Proof of GROW

		correctness.
5	**Engineering Fast Predecessor Search**	(Was Ch 4) Add Dinklage comparison; Proof of minpos correctness; Focus on *Small Set* niche.
6	**Dynamic Searchable Prefix Sums**	(Was Ch 5) Prove update consistency; Add case study (Memory Allocation).
7	**Evaluation & Case Studies**	**(New/Expanded)** Integrative benchmarks; Real-world scenarios (Cloud/Big Data).
8	**Conclusion**	Reflective analysis; Future directions (AVX-512).
App	**Appendices**	**(New)** Source Code snippets; UI Screenshots; Benchmark Raw Data.

6.2 The Passive Voice Transformation

The report notes the excessive use of "we" and "our".[1] This must be rigorously purged.

- *Before:* "We propose a range of dynamic data structures...".[1]
- *After:* "A range of dynamic data structures is proposed..."
- *Before:* "We intended to experiment further..."
- *After:* "Further experimentation is planned..."

This stylistic shift is crucial for meeting the "professional" tone required by the examiners.

---

7. Operational Roadmap: The 6-Month Plan

To deliver these major amendments, a strict schedule is required.

Phase 1: Foundations & Theory (Month 1-2)

- **Week 1-2:** Code Refactoring. Remove "we". Audit lines of code (LOC). Implement the UI Visualizer for Chapter 5 (a simple Python script reading memory dumps is sufficient).
- **Week 3-4:** Methodology Draft. Write Chapter 3. Define the machine specs. Formulate the RQs.
- **Week 5-8:** Theoretical Proofs. Write the "Verifiability Analysis" sections for Ch 4 & 5. Derive the Dynamic ITLB formulas for Ch 4.

Phase 2: Empirical Validation (Month 3-4)

- **Week 9-10:** Dinklage Benchmark. Setup tdc repo. Run head-to-head tests. Generate new plots.
- **Week 11-12:** Case Study Development. Create the "Big Data" scenario (e.g., compressing a mock Inverted Index). Collect data.
- **Week 13-16:** Writing the Evaluation Chapter. Integrate all new graphs. Ensure every graph has error bars (95% CI).

Phase 3: Synthesis & Final Polish (Month 5-6)

- **Week 17-20:** Full Thesis Restructuring. Move sections to fit the new chapter layout. Add summaries to every chapter.
- **Week 21-22:** Bibliography Audit. Check every reference for city/publisher.[1] Add Dinklage, Prezza, Pibiri citations.
- **Week 23-24:** Final Proofread. Check against the Examiner's Report line-by-line. Print and submit.

---

8. Detailed Technical Remediation Plans

8.1 Remediation of Chapter 3 (Extensible Arrays)

Problem: Need to justify contributions to the "Computing body of knowledge" and prove correctness.
Action:

1. **Theory:** Introduce the concept of "Bit-Aligned Resizing." Standard vectors resize by bytes/words. The contribution here is resizing by *bits* (or sub-word blocks).
2. **Proof:** Show that access(i) remains $O(1)$ despite the bit-misalignment caused by resizing.
   - *Proof Sketch:* $Access(i)$ reads word $\lfloor (i \times w)/64 \rfloor$. The offset is calculable in $O(1)$. The resizing preserves the relative bit-offset order.
3. **Data:** Include a table "Internal Fragmentation: std::vector vs. BitVectorEA." Show the % space saving.

8.2 Remediation of Chapter 4 (Predecessor Search)

Problem: Comparison with Dinklage and SIMD verification.
Action:

1. **Dinklage Analysis:** Add a section explicitly contrasting the "Universe Reduction" of Dinklage with the "SIMD Scan" of the thesis.
    - *Insight:* Dinklage is better for sparse, large universes. SIMD Scan is better for dense, small clusters (like B-tree leaves). This defines the *boundary* of the contribution.
2. **Verifiability:** Use the "XOR Min-Pos" proof derived in Section 3.1.2.
3. **Data:** The "Shift" experiment [1] needs a conclusion. *Why* does shifting reduce significant bits?
    - *Explanation:* Shifting aligns values to grid boundaries (powers of 2), increasing the length of the common prefix. This is a form of "delta encoding" alignment.

8.3 Remediation of Chapter 5 (Prefix Sum)

Problem: Clarify application and User Interface.
Action:

1. **Application:** Explicitly link to **Memory Management**. The "free list" in a memory allocator is a set of intervals. Finding a free block of size k is a "Searchable Prefix Sum" problem (finding index i where $Sum(i) \ge k$).
2. **User Interface (Prototype):** The examiner asked for a UI.
    - *Design:* Create a CLI tool that outputs the state of the C (buffer) and B (snapshot) arrays.
    - *Visual:* Generate a schematic diagram of the "Out-of-Sync" buffer accumulation.
    - *Inclusion:* Add "Figure 5.5: Visual representation of the Prefix Sum internal state during high-frequency updates."

8.4 New Chapter: Integration and Evaluation

The examiners want to see "real-world scenarios".1
Action: Construct a Synthetic Workload Generator.

- **Scenario:** Cloud Logging.
- **Data:** Streams of timestamps (integers).
- **Task:** Index the timestamps to allow "Range Count" queries (Predecessor Search) and "Data Volume" queries (Prefix Sum).
- **Result:** Compare the memory footprint of the Student's System vs. Apache Lucene (standard integer compression).

---

9. Conclusion

The path to a successful PhD defense is clear. The novelty of the work is not in question; the rigor of the presentation is. By systematically addressing the "Verification" and "Comparison" gaps—specifically by adopting formal proof sketches for the SIMD algorithms and benchmarking directly against Dinklage et al.—the thesis will meet the "Major Amendments"

criteria.

This plan transforms the thesis from a collection of implementations into a scientifically validated contribution to the field of Algorithm Engineering. The focus on quantified success criteria and formal correctness provides the necessary "Computing body of knowledge" justification required by the examiners.

Detailed Appendix: Implementation Specifications

A. Code Refactoring Guide (Passive Voice)

- **Pattern:** grep -r "we "
- **Replace:**
  - "We decided to use..." -> "The decision was made to utilize..."
  - "Our experiments show..." -> "Experimental results indicate..."
  - "We implemented..." -> "The algorithm was implemented..."

B. Bibliography Updates

- **Missing Fields:** Ensure every citation has: Author, Title, *Proceedings/Journal Name*, *Publisher*, *City*, *Year*, *Pages*, *DOI*.
- **New Citations:**
  - Dinklage, P., Fischer, J., & Herlez, A. (2021). *Engineering predecessor data structures for dynamic integer sets*. In *19th International Symposium on Experimental Algorithms (SEA 2021)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik.
  - Prezza, N. (2017). *A framework of dynamic data structures for string processing*. In *SEA*.
  - Pibiri, G. E., & Venturini, R. (2017). *Dynamic Elias-Fano representation*. In *CPM*.

C. Visualizer Requirement

- **Tool:** Python script (visualize_structure.py).
- **Input:** Binary dump from the C++ data structure.
- **Output:** SVG graphic showing the array blocks, the "gaps" (fragmentation), and the tree structure (for prefix sum).
- **Caption:** "Figure 5.X: Visualization of the Dynamic Prefix Sum structure, illustrating the synchronization buffer C overlaying the compressed array B."

D. Verification Logic Template (LaTeX)

For the thesis appendix:

Code snippet

```
\begin{theorem}
The SIMD Predecessor Search (Alg. 7) returns the index $i$ such that $LCP(S[i], x)$ is maximized.
\end{theorem}
\begin{proof}
Let $d_i = S[i] \oplus x$. The instruction \texttt{\_mm256\_minpos\_epu16} selects $k = \arg\min_i d_i$.
In lexicographical binary order, minimizing the XOR difference $d_i$ is equivalent to maximizing the position of the first distinguishing bit (from MSB down), which corresponds to the longest common prefix length.
Thus, $S[k]$ shares the longest prefix with $x$.
\end{proof}
```

This level of formalization is the "Silver Bullet" to satisfy the examiners' request for correctness proofs.

Works cited

1. Post-viva exam report and outcomes_ Examination - Abdulhakeem Ibrahim_ Examiner's report _ Haplo.pdf
2. Engineering Predecessor Data Structures for Dynamic Integer Sets - DROPS, accessed on January 14, 2026, https://drops.dagstuhl.de/storage/00lipics/lipics-vol190-sea2021/LIPIcs.SEA.2021.7/LIPIcs.SEA.2021.7.pdf
3. [2104.06740] Engineering Predecessor Data Structures for Dynamic Integer Sets - arXiv, accessed on January 14, 2026, https://arxiv.org/abs/2104.06740
4. Patrick Dinklage's research while affiliated with TU Dortmund University and other places, accessed on January 14, 2026, https://www.researchgate.net/scientific-contributions/Patrick-Dinklage-2123334308
5. Proving tree algorithms for succinct data structures - TuPL - Tufts Programming Languages, accessed on January 14, 2026, https://tupl.cs.tufts.edu/papers/1904.02809.pdf
6. Succinct data structure - Wikipedia, accessed on January 14, 2026,

https://en.wikipedia.org/wiki/Succinct_data_structure
7. Engineering Predecessor Data Structures for Dynamic Integer Sets - arXiv, accessed on January 14, 2026, https://arxiv.org/pdf/2104.06740
8. Engineering Predecessor Data Structures for Dynamic Integer Sets - DROPS, accessed on January 14, 2026, https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SEA.2021.7